Classification with Linear Models

Losses for linear classification, logistic regression, multiclass classification

Data Analytics and Mining, 2026

Fedor Shlyakhov
National Research University Higher School of Economics



November 13, 2026

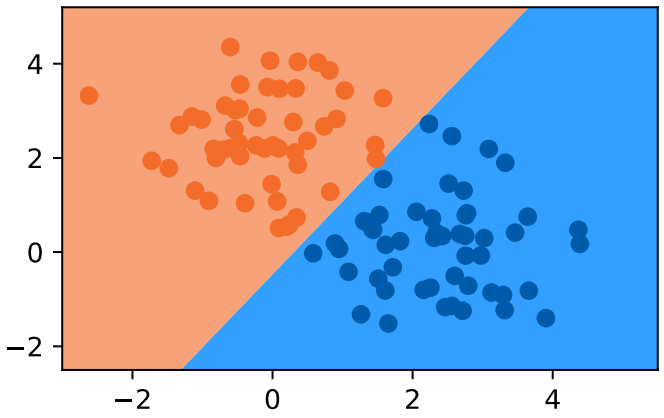
Can’t we just use linear regression for classification?



Classification with linear regression

Classification:

$$\hat{f}(x) = \text{sign}[\theta^{\mathrm{T}} x]$$

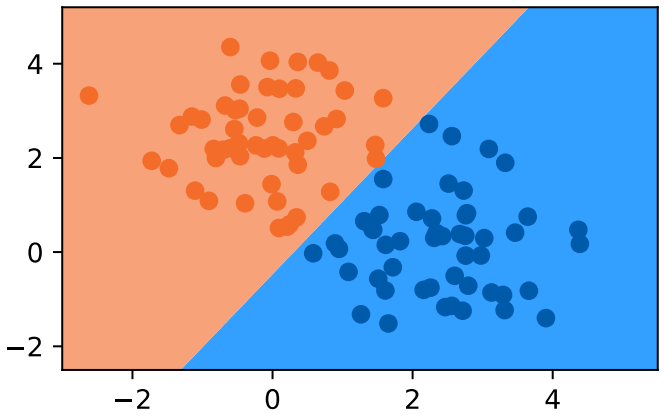


- For binary classification task, assign:
  - $y = +1$ for **positive** class
  - $y = -1$ for **negative** class

Classification with linear regression

Classification:

$$\hat{f}(x) = \text{sign}[\theta^{\mathrm{T}} x]$$

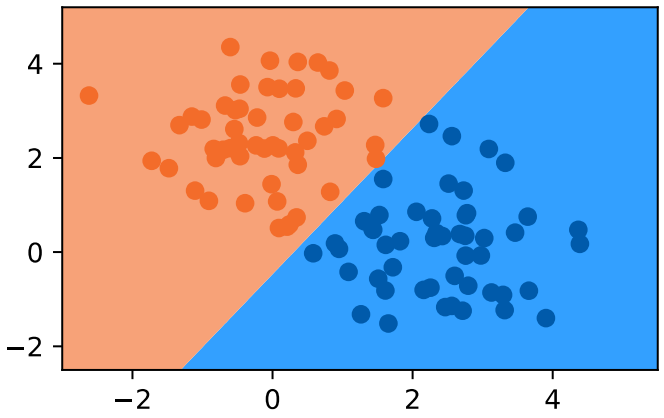


- For binary classification task, assign:
  - $y = +1$ for **positive** class
  - $y = -1$ for **negative** class
- Solve linear regression for $\hat{y} = \theta^T x$ with MSE loss

Classification with linear regression

Classification:

$$\hat{f}(x) = \text{sign}[\theta^{\mathrm{T}} x]$$

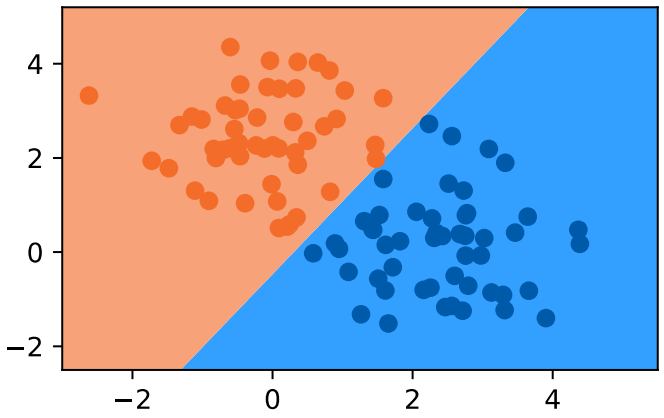


- For binary classification task, assign:
  - $y = +1$ for **positive** class
  - $y = -1$ for **negative** class
- Solve linear regression for $\hat{y} = \theta^T x$ with MSE loss
- Classify with $\text{sign}[\hat{y}]$

Classification with linear regression

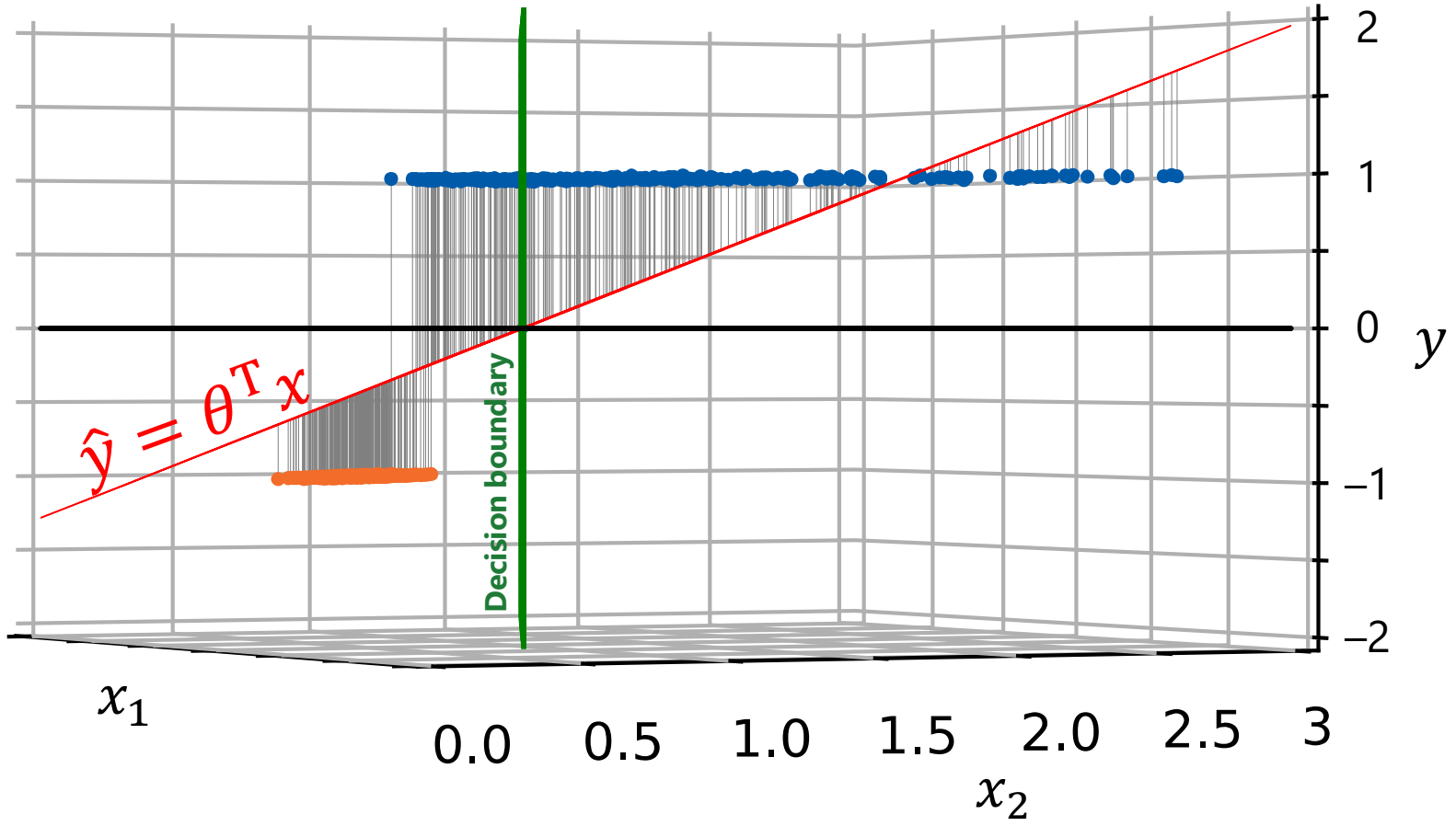
Classification:

$$\hat{f}(x) = \text{sign}[\theta^{\mathrm{T}} x]$$



- For binary classification task, assign:
  - $y = +1$ for **positive** class
  - $y = -1$ for **negative** class
- Solve linear regression for $\hat{y} = \theta^T x$ with MSE loss
- Classify with $\text{sign}[\hat{y}]$
- Any problems with this approach?

Classification with linear regression



MSE loss makes the model **avoid high residuals**

at a price of **pushing the decision boundary** towards the class with higher spread

Can we find a better loss function?

Classification loss functions

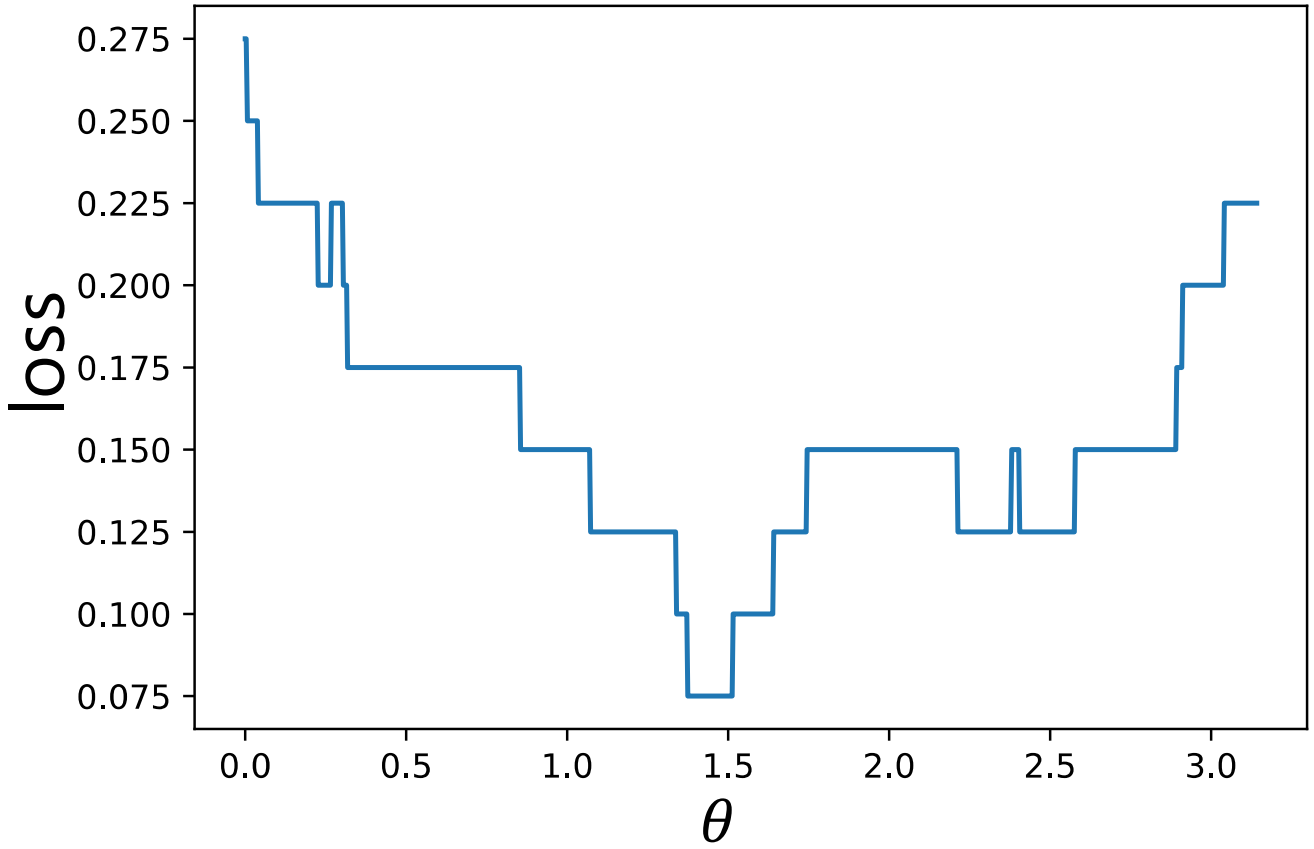


0-1 Loss

- Probability of an error

$$\mathcal{L}_{0-1} = \frac{1}{N} \sum_{i=1\ldots N} \mathbb{I}\left(\theta^{\mathrm{T}} x_i \cdot y_i < 0\right)$$

$$y_i \in \{-1, +1\}$$

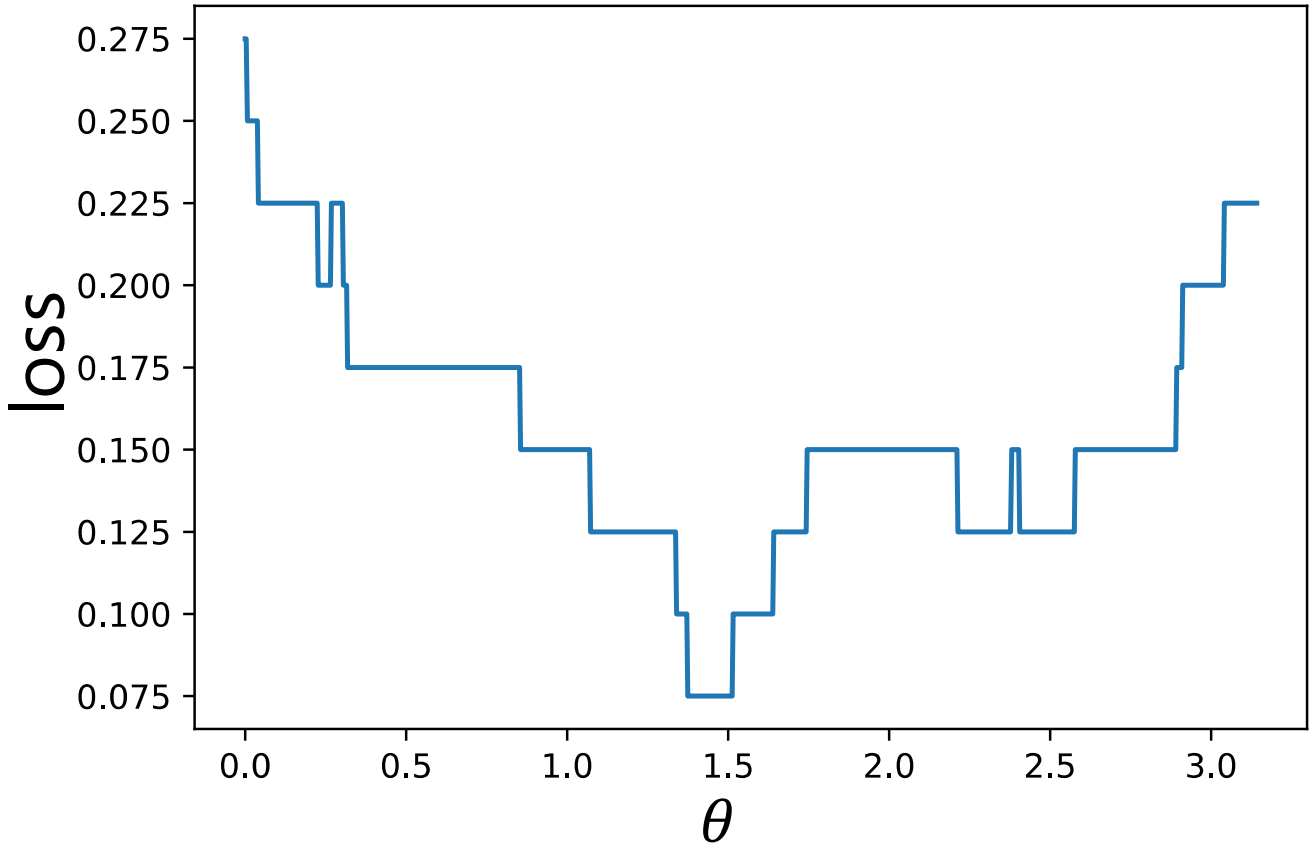


0-1 Loss

- Probability of an error

$$\mathcal{L}_{0-1} = \frac{1}{N} \sum_{i=1 \ldots N} \mathbb{I}\left(\theta^{\mathrm{T}} x_i \cdot y_i < 0\right)$$

$$y_i \in \{-1, +1\}$$



- Can't optimize **piecewise constant** function with gradient-based methods*

*other techniques exist (still quite limited)

Margin

$$M = \theta^{\mathrm{T}} x \cdot y$$

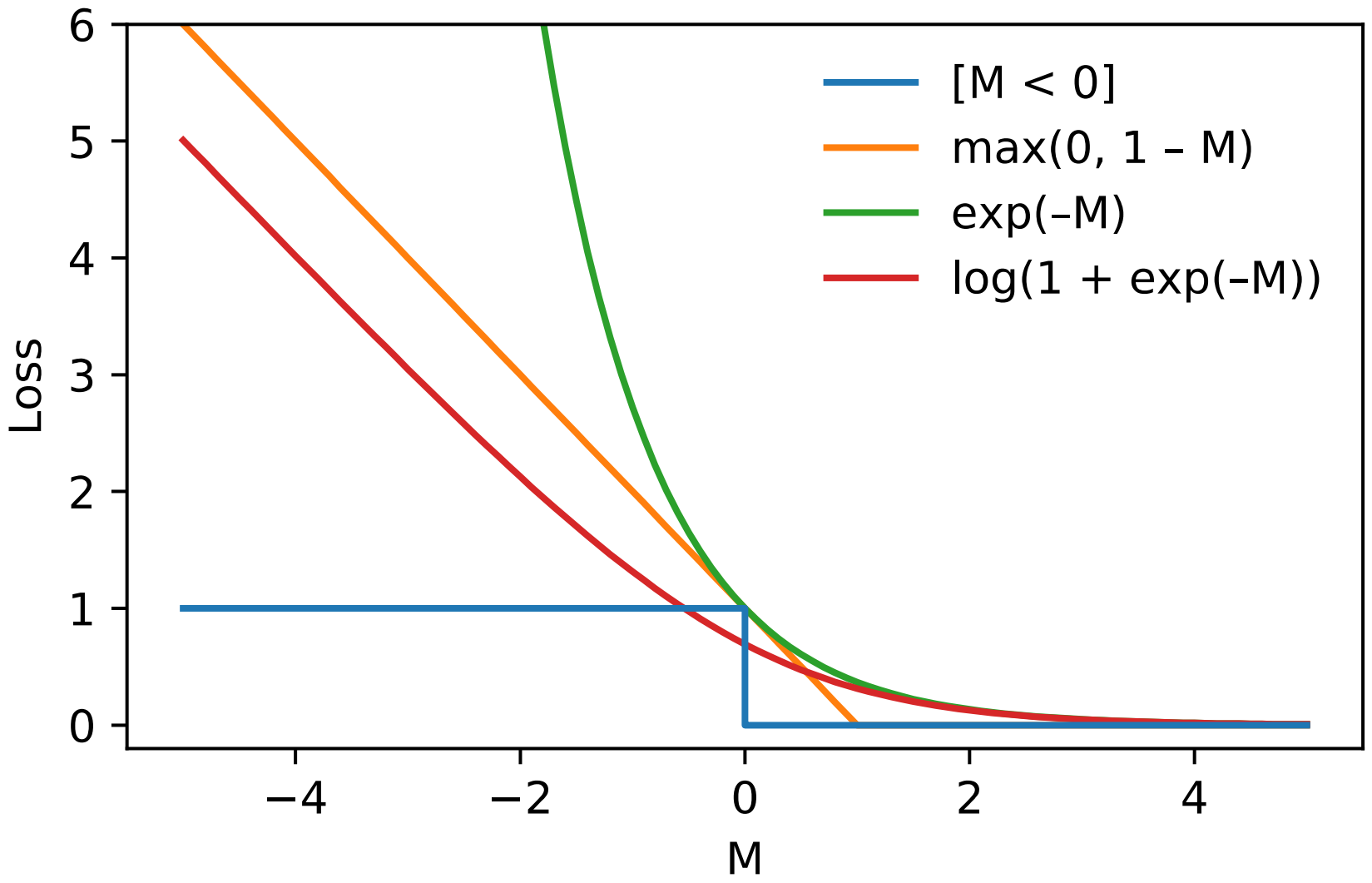
$$\mathcal{L}_{0-1} = \frac{1}{N} \sum_{i=1 \ldots N} \mathbb{I}\left(\theta^{\mathrm{T}} x_i \cdot y_i < 0\right)$$

margin

$M > 0$ – correct classification

$M < 0$ – incorrect classification

Upper bounds on 0-1 loss



Instead of optimizing the 0-1 loss we can optimize a **differentiable upper bound**

Learning check

1. $\theta^T x = -0.7$. The classifier $f(x) = \text{sign}[\theta^T x]$ predicts the class …

a. +1
b. −1
c. 0.7
d. 0

2. Why is the MSE loss a poor choice for classification?

a. it cannot be computed for labels ±1
b. it is always zero
c. it also punishes points that are far on the correct side, and this pushes the boundary
d. it needs probabilities as input

3. Margin $M = \theta^T x \cdot y$. An object has $y = -1$ and $\theta^T x = 2$. Then …

a. $M = -2$, the object is misclassified
b. $M = 2$, the object is classified correctly
c. $M = -2$, the object is classified correctly
d. $M = 2$, the object is misclassified

4. Why can we not minimize the 0-1 loss directly with gradient descent?

a. it can be negative
b. it is too large
c. it has no minimum
d. it is piecewise constant: its gradient is zero almost everywhere

Logistic Regression



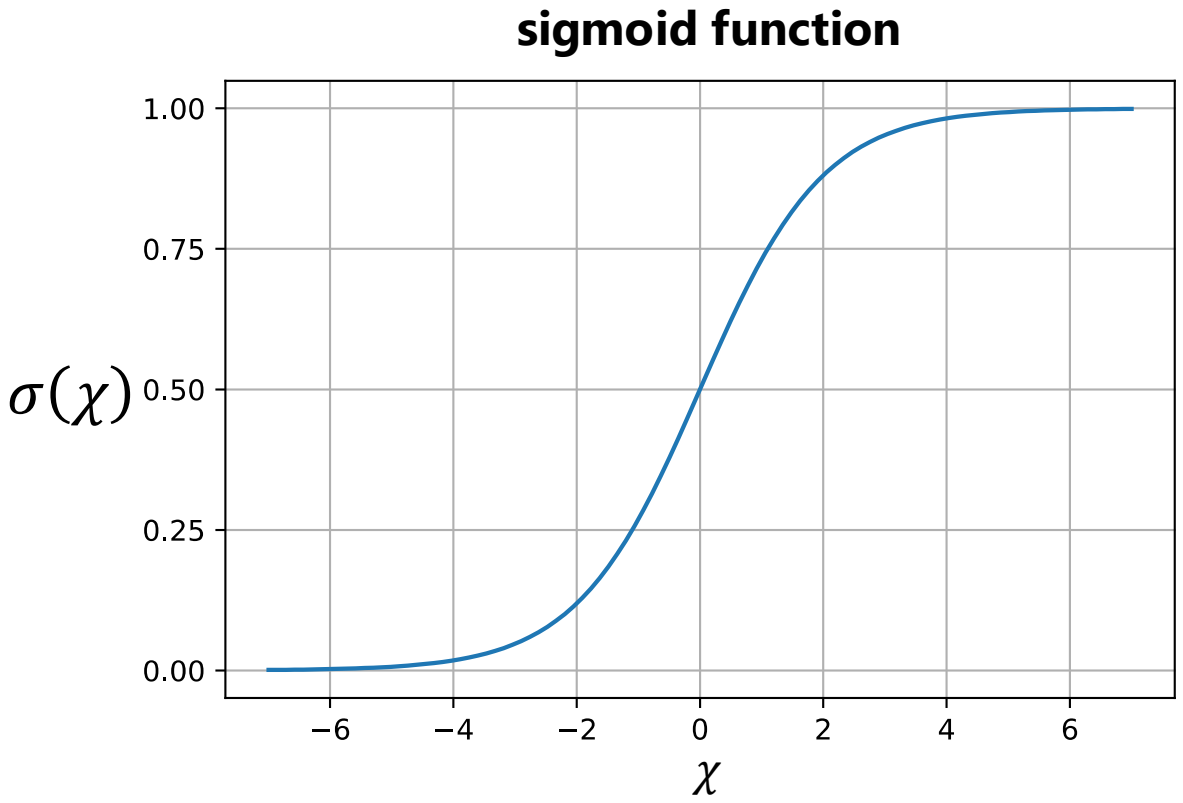
Linear probability model

- How to map the linear model output to a probability value in $[0, 1]$?

Linear probability model

- How to map the linear model output to a probability value in $[0, 1]$?
- Common choice – **sigmoid function**:

$$\sigma(\chi) = \frac{1}{1 + e^{-\chi}}$$

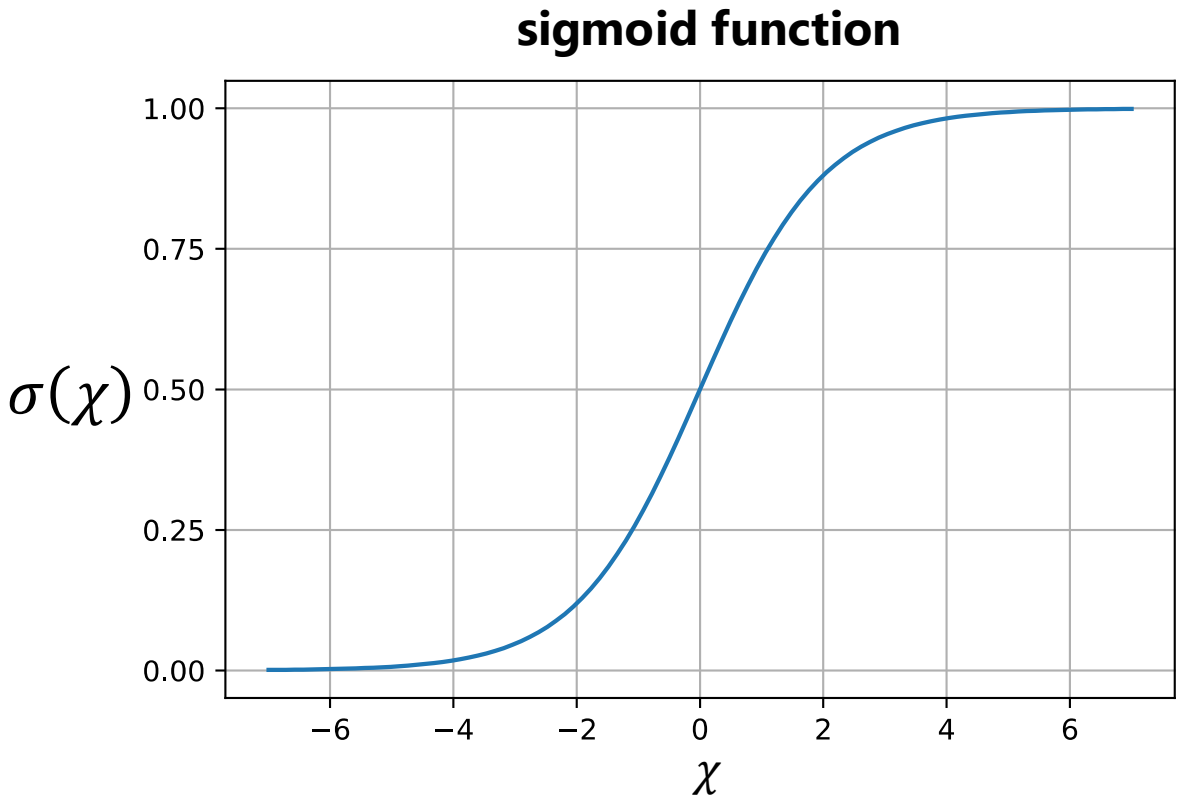


Linear probability model

- How to map the linear model output to a probability value in $[0, 1]$?
- Common choice – **sigmoid function**:

$$\sigma(\chi) = \frac{1}{1 + e^{-\chi}}$$

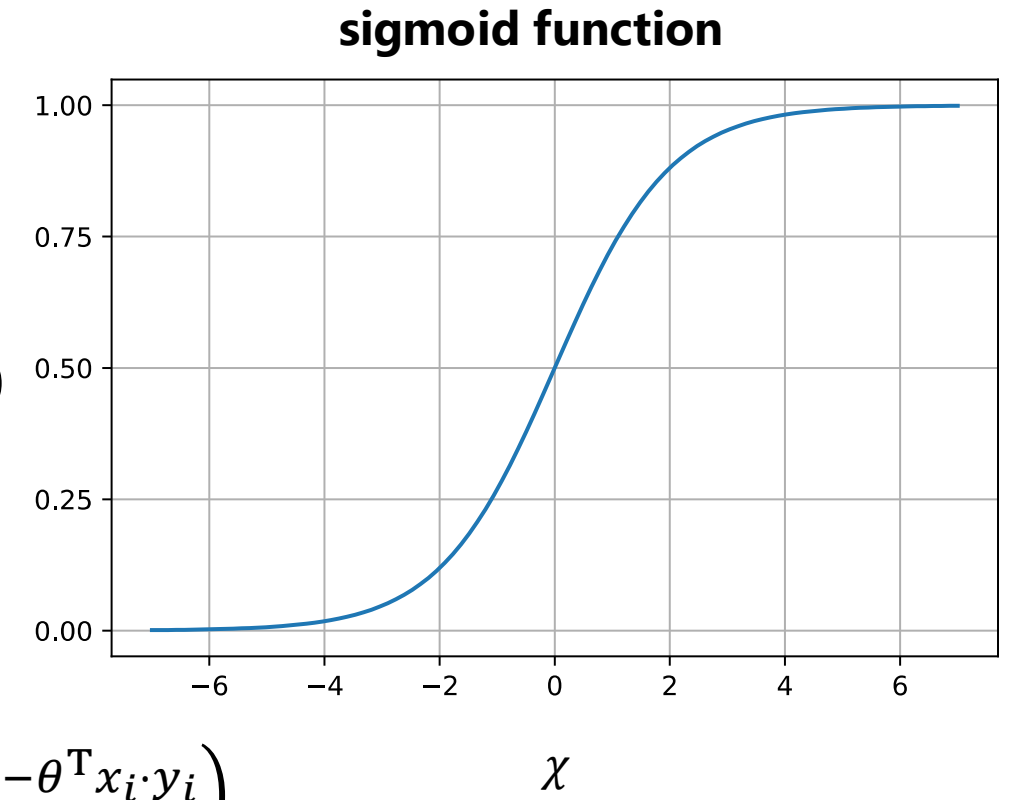
- I.e. $P(y = +1|x) = \sigma\left(\theta^{\mathrm{T}} x\right)$



Linear probability model

- How to map the linear model output to a probability value in $[0, 1]$?
- Common choice – **sigmoid function**:

$$\sigma(\chi) = \frac{1}{1 + e^{-\chi}}$$

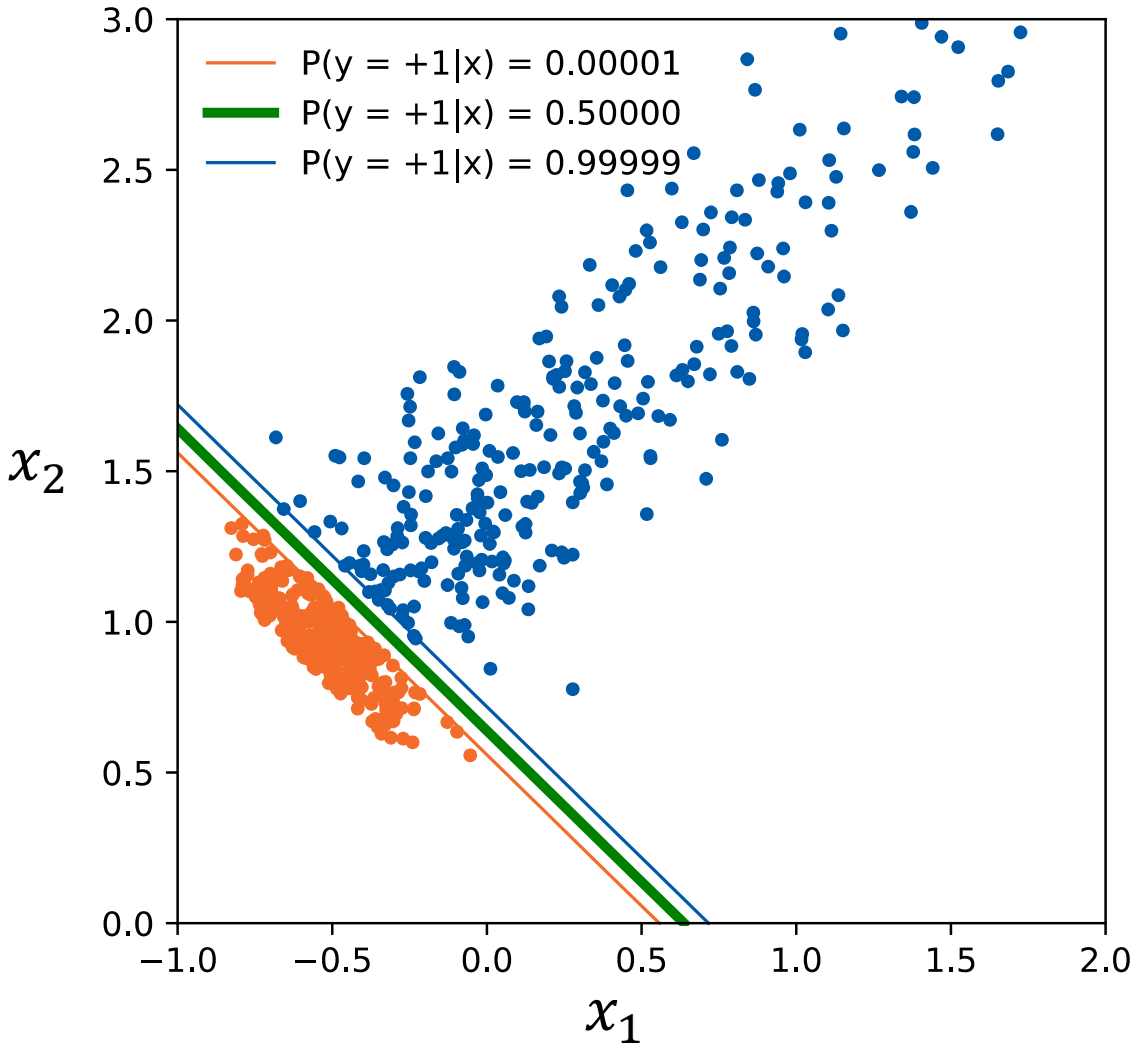


- Use negative log likelihood as our loss function:

$$\mathcal{L} = -\sum_{i=1\ldots N} \log \sigma\left(\theta^{\mathrm{T}} x_i \cdot y_i\right) = \sum_{i=1\ldots N} \log\left(1 + e^{-\theta^{\mathrm{T}} x_i \cdot y_i}\right)$$

- This can be optimized **numerically**

Example



- Now the boundary is at the right place

Learning check

1. The sigmoid $\sigma(x) = 1 / (1 + e^{-x})$. The value $\sigma(0)$ is …

a. 0
b. 1
c. 0.5
d. −1

2. $\theta^T x = 3$. Which value is closest to $P(y = +1 \mid x) = \sigma(3)$?

a. 0.95
b. 0.05
c. 0.5
d. 3

3. Logistic regression gives $P(y = +1 \mid x) = 0.8$. Then $P(y = -1 \mid x)$ is …

a. 0.8
b. 1.8
c. 0
d. 0.2

Multiclass Logistic Regression



Multinomial Logistic Regression

- Similarly to the binary case, we'll model the class probabilities
- Let's model **unnormalized** class probabilities like this:

$$\tilde{P}(y = k|x) = \exp \theta_k^{\mathrm{T}} x$$

Note: now we have K parameter vectors

Multinomial Logistic Regression

- Similarly to the binary case, we'll model the class probabilities
- Let's model **unnormalized** class probabilities like this:

$$\tilde{P}(y = k|x) = \exp \theta_k^{\mathrm{T}} x$$

Note: now we have K parameter vectors

- Then, the **normalized** probabilities are:

$$P(y = k|x) = \frac{\tilde{P}(y = k|x)}{\sum_{k'=1\ldots K} \tilde{P}(y = k'|x)} = \frac{\exp \theta_k^{\mathrm{T}} x}{\sum_{k'=1\ldots K} \exp \theta_{k'}^{\mathrm{T}} x}$$

  - This function is called **softmax** and is commonly used in neural networks

Multinomial Logistic Regression

- Plugging everything into the negative log likelihood we get our loss function:

$$\mathcal{L} = -\sum_{i=1\dots N} \log \frac{\exp \theta_{y_i}^{\mathrm{T}} x_i}{1 + \sum_{k'=1\dots K-1} \exp \theta_{k'}^{\mathrm{T}} x_i}$$

$$(\theta_K = 0)$$

- Again, this can be optimized **numerically**

Learning check

1. Three classes with $\exp(\theta_k^T x) = 1$, 1 and 2. Softmax gives class 3 the probability …

a. 2

b. 0.5

c. 0.33

d. 0.25

2. The softmax probabilities of all K classes always sum to …

a. 0

b. K

c. it depends on x

d. 1

Summary

- Classification with linear regression and MSE loss may provide **biased results**
- 0-1 loss function is better, but is **hard to optimize** directly
- Various **differentiable upper bounds** on 0-1 loss may be used instead
- Logistic Regression combines such an upper bound with a **probabilistic model** using the **sigmoid function**
- Generalizing sigmoid function to a multiclass case yields **softmax function**